

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа

Выполнил:

Попов Дмитрий

Группа

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача - разработка микросервисов для веб-приложения FRESH.

Ход работы

Для разработки микросервисной архитектуры веб-приложения FRESH были выполнены следующие шаги:

1. Проектирование микросервисов

Было выделено пять основных сервисов, каждый из которых отвечает за отдельную доменную область:

- User Service — управление пользователями, аутентификация, авторизация.
- Nutrition Service — дневник питания, расчёт КБЖУ, управление продуктами.
- Workout Service — дневник тренировок, управление упражнениями.
- Fasting Service — трекер интервального голодания.
- Progress Service — фотодневник и отслеживание прогресса.

2. Разработка API Gateway

Шлюз маршрутизирует входящие HTTP-запросы к соответствующим микросервисам. Конфигурация прокси динамическая, что позволяет легко добавлять новые сервисы. Базовые URL сервисов (например, `USER_SERVICE_URL`, `NUTRITION_SERVICE_URL`) хранятся в конфигурационном файле и загружаются из переменных окружения.

3. Реализация аутентификации на уровне шлюза

Разработан middleware, который перехватывает все входящие запросы (за исключением регистрации и логина) и валидирует JWT-токены. В случае успешной валидации из токена извлекается `userId`, который передаётся в заголовке `x-user-id` в целевые микросервисы. Это позволяет микросервисам идентифицировать пользователя без необходимости повторной проверки токена.

4. Разработка микросервисов

Внутри сервисов:

- Определены модели данных.
- Реализованы CRUD-операции через REST API.
- Настроено подключение к общей базе данных PostgreSQL с использованием отдельных схем для каждого сервиса.

Вывод

В ходе лабораторной работы была разработана микросервисная архитектура приложения FRESH, включающая пять специализированных сервисов, API Gateway с аутентификацией на уровне шлюза и механизмы межсервисного взаимодействия. Реализована единая обработка ошибок и обеспечена целостность данных при выполнении распределённых операций. Архитектура позволяет масштабировать приложение и поддерживать независимое развёртывание сервисов.